PROJETO INTEGRADOR DO 3° SEMESTRE
DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
– GRUPO 5

Testes e Qualidade de Software (DEVOPS) - Entrega 2

INTEGRANTES DO PROJETO e RA'S

Danilo Oliveira de Almeida - 23011087
Katiely de Jesus Silva - 23011355
Laura Piveta Pelizzer - 23011129
Matheus Marques Quio - 23011107

São Paulo
2026

SUMÁRIO

Introdução **3**
1. Qualidade de Software **3**
1.1. Modelo de Qualidade de Software (Diagrama/Design) 4
1.2. Processo (Plano) de Gerenciamento de Qualidade de Software 4
1.2.1. Metas de qualidade (derivadas do item 1.1) 5
1.2.2. Papéis e responsabilidades pela qualidade 6
1.2.3. Atividades de garantia da qualidade nas 4 atividades de Sommerville 7
1.2.4. Atividades preventivas 7
1.2.5. Critérios de aceitação / Definition of Done por incremento 8
1.3. Identificação de Atributos de Qualidade da Norma 25010 8
1.4. Relatório explicando como a norma 25010 é utilizada no desenvolvimento 9
2. Teste de Software **9**
2.1. Plano de Teste 9
2.2. Apresentar 4 Testes Unitários 10
2.3. Apresentar 2 Testes de Integração 13
2.4. Apresentar um Teste de Usuário (sistema/aceitação) 15
2.5. Relatório de Teste de Carga do Servidor 17
Apêndices **21**
Roteiro do Moderador: 21
Fala de abertura (ler em voz alta) 21
Tarefas (ler uma de cada vez, só passar para a próxima quando terminar) 21
Perguntas finais 22
Referências Bibliográficas **22**

Introdução

O Próxima Etapa é uma organização sem fins lucrativos cuja missão é democratizar o acesso à educação superior e à orientação profissional para jovens de escolas públicas. Como Projeto Integrador do 3º semestre do curso de Análise e Desenvolvimento de Sistemas da FECAP, o Grupo 5 está desenvolvendo o "App do Aluno": um aplicativo Android integrado a uma API REST e a um banco de dados, que permite ao estudante acompanhar cursos do próxima etapa, agenda/calendário, presença em palestras, certificados, testes de perfil e comunicação com a organização.

O problema que o sistema resolve é a dispersão de informações relevantes para o aluno atendido pelo Próxima Etapa, hoje distribuídas entre site institucional, redes sociais e contato direto, dificultando o acompanhamento de cursos, prazos e oportunidades. A proposta é centralizar essas informações em um aplicativo mobile simples, pensado para o público real da organização: jovens de 16 a 19 anos que, segundo a pesquisa de campo conduzida pelo grupo, raramente usam o site atual e chegam à Próxima Etapa majoritariamente pelo Instagram.

O público-alvo é o corpo de alunos da Próxima Etapa, e o objetivo do sistema é dar a esse público autonomia para acompanhar sua própria jornada educacional. As tecnologias adotadas são: aplicativo Android nativo com persistência local (SQLite/Room) no front-end; API REST em Node.js, Express e Prisma no back-end e Docker para conteinerização do ambiente de implantação.

O escopo desta entrega concentra-se no módulo do Aluno, preservando a estrutura de banco de dados já existente na Próxima Etapa. O módulo web administrativo previsto no edital será atendido pela camada de integração (API REST); as funcionalidades de pagamento não fazem parte do escopo. Este documento prepara o leitor para entender como qualidade e testes são aplicados ao longo do desenvolvimento: a Seção 1 aplica a norma ISO/IEC 25010 ao processo de qualidade do projeto, e a Seção 2 apresenta o plano de testes e os testes desenvolvidos. Nesta Entrega 2, a Seção 2 foi ampliada com os testes unitários e de integração revisados, os critérios do teste de sistema/aceitação (Seção 2.4) e o relatório de teste de carga do servidor (Seção 2.5).

---

1. Qualidade de Software

Para o App do Aluno, qualidade significa entregar as funcionalidades certas com desempenho, segurança, usabilidade, confiabilidade e manutenibilidade adequados ao contexto real da Próxima Etapa: um público jovem, conexão de internet nem sempre estável, e dados de estudantes que exigem cuidado sob a LGPD.

A qualidade do sistema é fundamentada na norma internacional ISO/IEC 25010, que define características e subcaracterísticas tanto para requisitos funcionais quanto não funcionais. O grupo já havia levantado requisitos não funcionais detalhados na Seção 5 do edital do projeto (separados por aplicativo mobile e por API/banco de dados); este documento reorganiza esses requisitos sob a lente da norma, definindo métricas e metas objetivas (Seção 1.3) e um processo com pontos de controle explícitos (Seção 1.2) para medir e garantir essa qualidade ao longo do desenvolvimento.

1.1. Modelo de Qualidade de Software (Diagrama/Design)

Característica	Exemplo no App do Aluno
Adequação Funcional	● *Correção funcional:* a agenda exibe apenas os encontros dos cursos em que o aluno está inscrito, com carga horária e nº de encontros corretos
Capacidade de Interação	● *Operabilidade e Inclusividade:* check-in por QR Code concluído em poucos toques (< 10 s), com contraste mínimo de 4,5:1 e rótulos de acessibilidade
Confiabilidade	● *Tolerância a falhas:* presença registrada sem internet fica salva localmente (Room) e é sincronizada automaticamente ao reconectar
Segurança (Security)	● *Confidencialidade:* cada aluno autenticado (JWT) acessa somente os próprios dados, com toda a comunicação via HTTPS

A Tabela apresenta o modelo hierárquico de qualidade adotado, com as 4 características principais do Modelo de Qualidade do Produto da ISO/IEC 25010:2023 e suas subcaracterísticas. Essas características foram priorizadas por concentrarem os requisitos não funcionais mais críticos e explícitos do edital: completude das funcionalidades do módulo Aluno, interface simples e acessível para um público jovem, tolerância a falhas de conexão e proteção de dados via HTTPS e conformidade com a LGPD. Para cada uma delas, a terceira coluna traz um exemplo concreto de como a característica aparece no App do Aluno.

As demais características, Eficiência de Desempenho, Compatibilidade, Manutenibilidade e Flexibilidade, seguem relevantes ao projeto e possuem metas próprias na Seção 1.2, mas não compõem a matriz de rastreabilidade dos 4 atributos da Seção 1.3. A característica Proteção (Safety), introduzida na versão de 2023, tem aplicabilidade limitada a um aplicativo educacional, que não controla riscos físicos ou ambientais; no projeto, ela se manifesta apenas na apresentação responsável dos resultados do teste de perfil, que não devem ser interpretados como diagnóstico psicológico, conforme exigido pelo edital.

1.2. Processo (Plano) de Gerenciamento de Qualidade de Software

Objetivo: garantir que cada incremento do App do Aluno atenda às metas de qualidade derivadas da ISO/IEC 25010 antes de ser entregue, por meio de pontos de controle definidos no processo de desenvolvimento.

Escopo: o plano abrange o aplicativo Android (módulo Aluno), a API REST e o banco de dados, cobrindo todos os ciclos do projeto.

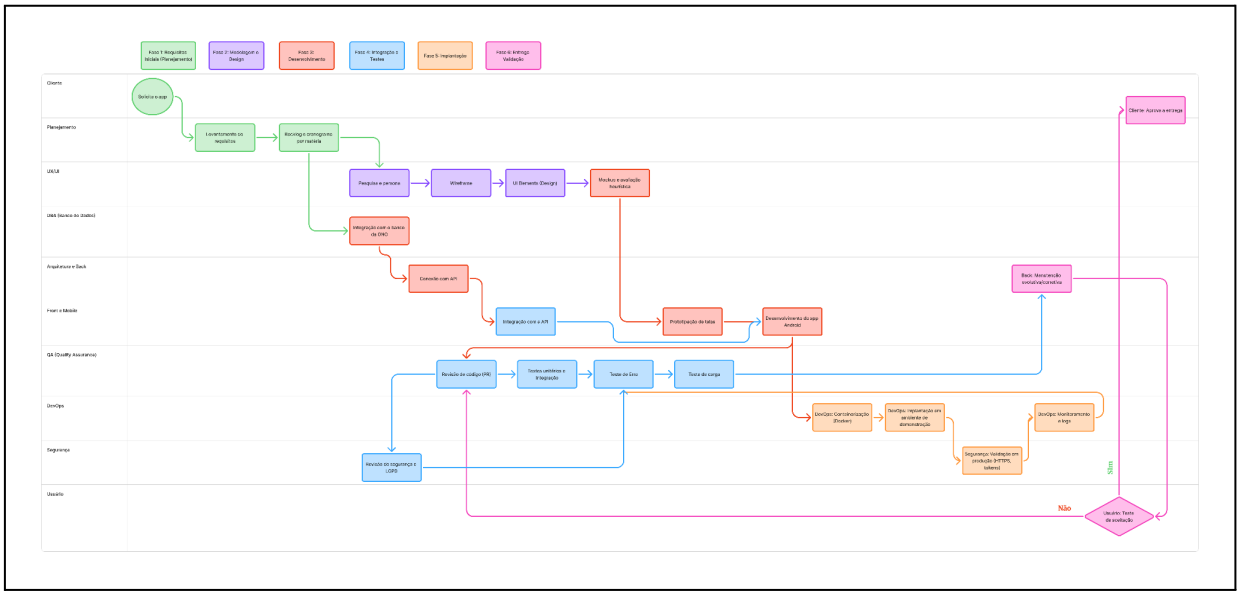


Figura 1 – Diagrama do processo de qualidade de software do App do Aluno

Link do Figma:

https://www.figma.com/board/KeR0g27Nng7htY4xN0HblX/Processo-de-Qualidade---Ciclo-Completo-por-Frente?node-id=0-1&t=Usdvh7x6xeBqvhpV-1

A Tabela ilustra o fluxo adotado com dois pontos de controle de qualidade (revisão de código e bateria de testes) que impedem que código não conforme avance para a etapa seguinte.

O processo é organizado em seis fases (Planejamento, Modelagem e Design, Desenvolvimento, Revisão e Testes, Implantação e Entrega) e em raias por papel (Cliente, Planejamento, UX, Back-end, Mobile, QA, DevOps e Segurança). O código só avança para a fase de Teste após aprovação no Pull Request e só é entregue após passar pela bateria de testes; em caso de reprovação, retorna ao Desenvolvimento.

1.2.1. Metas de qualidade (derivadas do item 1.1)

Característica	Métrica	Meta
Adequação Funcional	% de RFMs (01–15) implementados e cobertos por critério de aceitação	100% até a Entrega 2
Segurança	Nº de endpoints servidos fora de HTTPS ou com queries não parametrizadas	0
Confiabilidade	% de registros de presença sincronizados corretamente após reconexão	100%

Característica	Métrica	Meta
Capacidade de Interação	Taxa de conclusão das tarefas-chave (QR Code, inscrição em palestras/cursos, editar perfil, acessar calendário e eventos) em teste de usabilidade	≥ 90% dos participantes concluem sem ajuda
Eficiência de Desempenho	Tempo de resposta das consultas/listagens sob carga simulada	< 2s
Compatibilidade	% dos testes de integração app ↔ API passando	100%
Manutenibilidade	% de pull requests mesclados com revisão de código aprovada	100%
Flexibilidade	Ambiente (API + banco) sobe com docker compose up em máquina limpa, sem passos manuais	Em até 10 min
Proteção (Safety)	% das telas de resultado do teste de perfil com aviso de "orientação, não diagnóstico psicológico"	100%

1.2.2. Papéis e responsabilidades pela qualidade

Papel	Responsabilidade pela qualidade
Planejamento (todo o time)	Definir requisitos do MVP para apresentação do trabalho e também features extras se der tempo; e critérios de aceitação de cada incremento no GitHub Project
Mobile (Katiely)	Implementar as telas seguindo a arquitetura definida, com código comentado; corrigir os apontamentos da revisão
QA (Katiely/Laura)	Revisar Pull Requests (aprova ou reprova); escrever e executar os testes unitários, de integração e de carga; verificar se as metas de qualidade foram atingidas antes de cada entrega
Back-end / DBA (Katiely)	Implementar a API em camadas com validação de entrada e queries parametrizadas; garantir a integridade do banco (chaves, restrições e presença única por aluno + atividade + data/horário)
Cloud Native (Danilo)	Automatizar backup via script, containerizar com Docker, implantar em ambiente de demonstração e monitorar logs

Papel	Responsabilidade pela qualidade
DevOps - Teste e Qualidade de Software (Laura)	Conduzir o processo de qualidade: manter o diagrama e as metas da ISO/IEC 25010, planejar e consolidar os testes e o relatório de carga, e verificar a Definition of Done antes de cada entrega.
Segurança (Katiely)	Aplicar o checklist de LGPD (coleta mínima, sem dados reais em repositório ou demo) e revisar autenticação/HTTPS antes de cada implantação
Design e UX (Quio)	Produzir wireframes e mockups, aplicar heurísticas de usabilidade e acessibilidade, e conduzir os testes de usabilidade que medem a Capacidade de Interação.
Cliente (Próxima Etapa)	Validar e aprovar a entrega final de cada ciclo

1.2.3. Atividades de garantia da qualidade nas 4 atividades de Sommerville

Atividade (Sommerville)	O que o grupo faz
Especificação	Levantamento das features, backlog no GitHub Project e critérios de aceitação por matéria - etapa Planejamento
Projeto e Implementação	Arquitetura em camadas (app Android, API Node + Express, banco Supabase; SQLite/Room apenas no armazenamento local do app), desenvolvimento em branches por funcionalidade e revisão de código em Pull Request - etapas Desenvolvimento e Revisão
Validação	Testes unitários, de integração, de carga, de segurança e de aceitação, comparando os resultados com as metas de qualidade - etapa Teste
Evolução	Manutenção corretiva e evolutiva a partir do feedback do cliente e dos professores; o que surgir volta para o backlog do próximo ciclo - etapa Entrega

1.2.4. Atividades preventivas

- Revisão de requisitos e critérios de aceitação no Planejamento, antes de qualquer código ser escrito. Isso evita retrabalho por requisito mal entendido.
- Checklist de code review (padrão de código, nomenclatura, tratamento de erro, queries parametrizadas) obrigatório em todo Pull Request antes do merge, na etapa Revisão do processo.

- Uso exclusivo de dados fictícios ou anonimizados em desenvolvimento, testes e demonstrações. Isso previne exposição de dados pessoais, conforme a LGPD e a Seção 5.2 do edital.

1.2.5. Critérios de aceitação / Definition of Done por incremento

- Funcionalidade implementada conforme o RFM correspondente, sem erros conhecidos.
- Código revisado e aprovado em Pull Request por outro integrante do grupo.
- Teste (unitário e/ou de integração) relacionado escrito e passando.
- Requisito não funcional aplicável (Seção 5 do edital) verificado — ex.: HTTPS, tratamento de erro, contraste e rótulos de acessibilidade.
- Nenhum dado pessoal real no código, no repositório ou na demonstração.
- Documentação mínima atualizada (README ou comentário relevante no código).

1.3. Identificação de Atributos de Qualidade da Norma 25010

A tabela a seguir é a matriz de rastreabilidade dos 4 atributos priorizados na Seção 1.1, ligando um requisito/cenário real do projeto à característica e subcaracterística da norma, à métrica que o mede, à meta definida e à forma de verificação.

Requisito / Cenário	Característica (25010)	Subcaracterística	Métrica	Meta	Como verificar
Consulta e listagem de cursos/agenda	Adequação Funcional	Correção Funcional	% de RFMs 01–15 testados e aprovados	100% até E2	Testes de integração + checklist de aceitação por RFM
Registro de presença via QR Code	Capacidade de Interação	Operabilidade	Tempo médio para concluir o check-in	< 10s	Teste de usabilidade com usuários reais
Sincronização após perda de conexão	Confiabilidade	Tolerância a Falhas	% de registros sincronizados corretamente após reconexão	100%	Teste manual em modo avião + reconexão
Autenticação e comunicação com a API	Segurança	Confidencialidade / Integridade	Nº de endpoints fora de HTTPS ou com queries não parametrizadas	0	Revisão de configuração do servidor + code review

---

1.4. Relatório explicando como a norma 25010 é utilizada no desenvolvimento

Na etapa de Planejamento, a norma entra antes de qualquer código ser escrito: os requisitos não funcionais da Seção 5 do edital são reorganizados como características e subcaracterísticas da ISO/IEC 25010, e cada um dos 4 atributos priorizados recebe uma métrica e uma meta objetiva (Seção 1.3), que se tornam parte do critério de aceitação do incremento.

Na etapa de Desenvolvimento, a arquitetura em camadas (controller/service/repository na API; Activities/Fragments organizados por responsabilidade no app) é uma decisão direta da característica Manutenibilidade — especificamente das subcaracterísticas Modularidade e Testabilidade —, pensada para que os testes unitários e de integração da Entrega 2 tenham componentes isoláveis para testar.

Na etapa de Revisão, o checklist de Pull Request verifica, além de padrão de código, pontos de Segurança (uso de queries parametrizadas, ausência de credenciais no código) e de Adequação Funcional (aderência ao RFM correspondente) — a norma vira, nesse momento, um checklist objetivo, não apenas um conceito teórico.

Na etapa de Teste, a norma se traduz diretamente nos tipos de teste exigidos pela Entrega 2 desta Unidade Curricular: os testes unitários e de integração verificam Adequação Funcional; o relatório de teste de carga verifica Eficiência de Desempenho; e testes específicos de reconexão e autenticação verificam, respectivamente, Confiabilidade e Segurança, e o teste de usabilidade com alunos da Próxima Etapa verifica Capacidade de Interação, em especial a operabilidade do check-in por QR Code.

Na etapa de Entrega, o teste de sistema/aceitação (Seção 2.4) verifica o produto como um todo em relação a todos os atributos priorizados simultaneamente — não apenas às funcionalidades previstas nos RFMs —, fechando o ciclo de aplicação da norma até a Apresentação Final.

---

2. Teste de Software

Os testes são a forma prática de verificar, com evidência, que o sistema atende às metas de qualidade definidas na Seção 1. O grupo adota testes em três níveis: unitário (componentes isolados), integração (componentes conectados, ex.: API e banco de dados) e usuário/sistema (o produto completo, do ponto de vista de quem usa). Os testes unitários e de integração, junto com o relatório de teste de carga, são entregáveis obrigatórios da Entrega 2 (06/11/2026) desta Unidade Curricular; o teste de sistema/aceitação também integra a Entrega 2, conforme o edital, e é demonstrado novamente na Apresentação. O relatório de teste de carga está na Seção 2.5.

2.1. Plano de Teste

Tipo de teste	Ferramenta	Responsável	Momento (cronograma)
Testes unitários	Jest com mocks dos repositórios (sem banco e sem rede)	QA	26/10–30/10

Tipo de teste	Ferramenta	Responsável	Momento (cronograma)
Testes de integração	Jest + Supertest contra banco Supabase de teste isolado (Docker)	QA	31/10–03/11
Teste de carga do servidor	k6 contra a API em Docker (Seção 2.5)	QA / DevOps	03/11–05/11
Teste de sistema / aceitação	Roteiro manual com usuário real	QA + Usuário	04/11–05/11 (registro na Entrega 2) e Apresentação (17–24/11)

Critérios de sucesso, falha e correção

- Sucesso: o teste passa sem intervenção manual e o resultado obtido é igual ao resultado esperado.
- Falha: há divergência entre resultado obtido e esperado, exceção não tratada, ou timeout.
- Procedimento de correção: abrir uma tarefa referenciando o teste que falhou, corrigir na branch de desenvolvimento e reexecutar o teste antes de abrir novo Pull Request.

2.2. Apresentar 4 Testes Unitários

Foram desenvolvidos quatro testes unitários automatizados para componentes centrais do back-end da solução, implementado em Node.js com Express e TypeScript. Os testes utilizam o framework Jest e isolam cada componente de suas dependências externas, como banco de dados e requisições HTTP, por meio de objetos simulados (mocks), de modo que cada teste verifique uma única unidade de lógica.

Os componentes foram escolhidos por estarem ligados a regras de negócio e requisitos críticos do briefing da Próxima Etapa: a unicidade da presença por aluno, atividade e encontro (seção 3 do briefing), o dado numérico de participação do aluno, o registro de presença por QR Code e a autenticação segura de acesso à API.

Ambiente e execução (lista curta)

Linguagem: TypeScript 5, executado em Node.js.

Framework de testes: Jest 29, com ts-jest.

Bibliotecas: Express (API) e jsonwebtoken (tokens JWT de autenticação e de QR Code).

Execução: na raiz da pasta, npm install para instalar as dependências e npm test para rodar os testes.

Repositório:
https://github.com/2026-2-NADS3/Projeto5/tree/main/documentos/Entrega%202/Teste%20e%20Qualidade%20de%20software%20(DevOps)/testes-unitarios

Para o teste de validação do QR Code, o grupo definiu que o conteúdo do código será um token JWT assinado pelo servidor, contendo o identificador da atividade e um prazo de validade de 15 minutos. Dessa forma, um QR Code só é aceito se a assinatura corresponder à chave do servidor e o prazo não tiver expirado, o que impede a geração de códigos falsos e o reaproveitamento de códigos antigos.

Descrição e resultados dos testes

O Quadro X apresenta cada teste unitário, com o componente testado, o requisito relacionado, o objetivo, a entrada utilizada, o resultado esperado e o resultado obtido na execução.

Testes unitários do back-end

Nº	Componente / função testada	Objetivo	Entrada utilizada	Resultado esperado	Resultado obtido
1	Serviço de presença (PresencaService.registrar), com repositório simulado (mock) Requisito: RFM06 e regra de negócio da seção 3	Impedir presença duplicada no mesmo encontro	Aluno fictício ID 1, atividade ID 10, encontro 15/10/2026 19h, já registrado	Lança o erro "presença já registrada"; o método de gravação do repositório não é chamado	Aprovado. O serviço lançou o erro "presença já registrada" e o método salvar() do repositório simulado não foi chamado. Cenário complementar: sem registro prévio, a presença foi gravada uma única vez.
2	Cálculo do percentual de participação (calcularPercentualParticipacao) Requisito: RFM07	Garantir o cálculo correto do dado numérico de participação	6 presenças em 8 encontros; e 0 encontros	75%; e 0% sem erro de divisão por zero	Aprovado. A função retornou 75 para 6 presenças em 8 encontros e 0 para 0 encontros, sem lançar exceção.
3	Validação do QR Code (validarQrCode) Requisito: RFM06	Aceitar apenas QR Codes válidos e dentro do prazo	Token válido; token expirado; token com assinatura alterada	Válido retorna o ID da atividade; os outros dois são rejeitados	Aprovado. O token válido retornou o ID da atividade (10); o token expirado e o token com assinatura alterada retornaram null e foram rejeitados.

Nº	Componente / função testada	Objetivo	Entrada utilizada	Resultado esperado	Resultado obtido
4	Middleware de autenticação JWT da API (autenticar), com requisição e resposta simuladas Requisito: RFM01 e RNF de segurança (5.1)	Bloquear acesso sem autenticação válida	Requisição sem token; com token inválido; com token válido	401 nos dois primeiros casos; acesso liberado com o aluno identificado no terceiro	Aprovado. Sem token e com token inválido, a resposta foi 401 e a requisição não prosseguiu; com token válido, o acesso foi liberado e o aluno foi identificado (req.aluno = { id: 1 }).

Cada teste unitário cobre mais de um cenário, positivo e negativo, conforme as entradas descritas no na tabela acima. Por esse motivo, os quatro testes correspondem a dez casos de teste executados, distribuídos conforme o quadro abaixo

Casos de teste executados por teste unitário

Nº	Teste	Cenários (casos de teste)	Casos	Situação
1	Serviço de presença	Presença duplicada; presença nova	2	2 aprovados
2	Percentual de participação	6 de 8 encontros; 0 encontros	2	2 aprovados
3	Validação do QR Code	Token válido; expirado; assinatura alterada	3	3 aprovados
4	Middleware JWT	Sem token; token inválido; token válido	3	3 aprovados
	Total		10	10 aprovados, 0 falhas

Análise dos resultados

Os resultados confirmam que as regras críticas da solução estão protegidas no nível da unidade. O serviço de presença impede registros duplicados para o mesmo encontro antes de qualquer gravação no banco; o cálculo de participação trata o caso-limite de zero encontros sem erro; QR Codes expirados ou com assinatura adulterada são rejeitados, o que impede fraude no check-in; e as rotas protegidas da API retornam o código 401 para requisições não autenticadas.

Esses testes contribuem diretamente para as características de Adequação Funcional e Segurança da norma ISO/IEC 25010, mapeadas no relatório de qualidade da Entrega 1, e para a Manutenibilidade (subcaracterística testabilidade), já que os componentes foram estruturados com dependências injetáveis justamente para permitir testes isolados.

Como próximo passo, a interface de repositório do serviço de presença será implementada com o Prisma sobre o banco Supabase, com uma restrição de unicidade sobre aluno, atividade e encontro, reforçando a mesma regra também no banco de dados. Os testes de integração, apresentados na seção seguinte, validam a comunicação real entre a API e o banco.

2.3. Apresentar 2 Testes de Integração

Foram desenvolvidos dois testes de integração automatizados que verificam o funcionamento conjunto da API REST e do banco de dados Supabase. Diferentemente dos testes unitários, aqui nenhuma dependência é simulada: cada teste envia requisições HTTP reais à API, por meio da biblioteca Supertest, e a API lê e grava em um banco de dados de teste, verificando tanto a resposta recebida quanto o estado final do banco.

Os cenários escolhidos correspondem aos dois fluxos principais da Entrega 2: o acesso autenticado aos dados do próprio aluno (login e consulta de cursos) e o registro de presença por QR Code com consulta ao histórico, que concentra a regra de negócio de presença única por encontro.

Ambiente e execução

Framework: Jest 29 com Supertest, em TypeScript.

API: Node.js com Express, com as rotas POST /auth/login, GET /alunos/me/cursos, POST /presencas e GET /presencas/historico.

Banco de dados: Supabase, executado em contêiner pelo arquivo docker-compose.yml (porta 5433).

Dados: somente dados fictícios, inseridos automaticamente antes de cada suíte (dois alunos, três cursos e sete encontros), em conformidade com a LGPD e com o requisito de não usar dados reais em testes.

Execução: docker compose up -d para subir o banco e npm run test:integration para rodar os testes.

Repositório:
https://github.com/2026-2-NADS3/Projeto5/tree/main/documentos/Entrega%202/Teste%20e%20Qualidade%20de%20software%20(DevOps)/teste-integracao

A restrição de unicidade da presença foi implementada diretamente no banco, com a cláusula UNIQUE (aluno_id, encontro_id) na tabela de presença. Assim, mesmo que duas requisições de check-in cheguem ao mesmo tempo, o banco impede o registro duplicado. O acesso aos dados nesta versão usa o driver node-supabase com consultas parametrizadas; o schema Prisma equivalente já está no repositório, e os testes continuam válidos após a migração para o Prisma, pois verificam a API pelo protocolo HTTP e o resultado gravado no banco.

Descrição e resultados dos testes

A tabela apresenta cada teste de integração, com o cenário, os componentes envolvidos, o fluxo executado, o resultado esperado e o resultado obtido.

Testes de integração da API com o banco de dados

Nº	Cenário	Componentes envolvidos	Fluxo executado	Resultado esperado	Resultado obtido
1	Login e consulta de cursos do aluno Requisitos: RFM01, RFM03 e RNF de privacidade (5.1)	API (autenticação + cursos), camada de acesso a dados, banco Supabase de teste	POST /auth/login com aluno fictício → recebe token → GET /alunos/me/cursos com o token	HTTP 200 com apenas os cursos em que esse aluno está inscrito; nenhum dado de outro aluno retornado	Aprovado. O login retornou 200 com token; a consulta retornou 200 com apenas os cursos 1 e 2, em que a aluna fictícia está inscrita. O curso 3, exclusivo do outro aluno, não apareceu, e a resposta não trouxe e-mail nem dado pessoal de nenhum aluno. Cenário complementar: senha incorreta e consulta sem token retornaram 401.
2	Check-in por QR Code ponta a ponta Requisitos: RFM06, RFM07 e regra de negócio da seção 3	API (presença + histórico), camada de acesso a dados, banco Supabase de teste	POST /presencas com QR válido do encontro → repetir a mesma requisição → POST /presencas com QR expirado → GET /presencas/historico	201 na primeira; 409 na repetição, com uma única linha no banco; 400 no QR expirado; histórico mostra a presença e o percentual atualizado	Aprovado. A primeira requisição retornou 201; a repetição retornou 409 e a consulta direta ao banco encontrou uma única linha; o QR expirado retornou 400 sem gravar nada; o histórico retornou a presença do encontro 1 e o percentual de 25% (1 presença em 4 encontros) no curso.

Casos de teste executados por teste de integração

Nº	Teste de integração	Casos de teste	Situação
1	Login e consulta de cursos	Fluxo principal (login + cursos); credenciais inválidas e acesso sem token	2 aprovados
2	Check-in por QR Code	Fluxo completo: 201 → 409 → 400 → histórico	1 aprovado
	Total		3 aprovados, 0 falhas

Para confirmar que o teste 2 de fato depende da regra do banco, a restrição UNIQUE foi removida temporariamente e o teste foi executado novamente: a requisição repetida passou a retornar 201 em vez de 409, e o teste falhou. Com a restrição restabelecida, o teste voltou a ser aprovado. Isso demonstra que o teste é capaz de detectar a falha que se propõe a verificar.

Análise dos resultados

Os resultados confirmam que os componentes funcionam corretamente em conjunto. No primeiro teste, o token gerado no login é aceito pelas rotas protegidas e a consulta retorna apenas os dados do aluno autenticado, atendendo ao requisito de privacidade de que cada estudante visualize somente os próprios dados. No segundo, o fluxo completo de check-in mostra que a validação do QR Code, a gravação no banco, a restrição de unicidade e o cálculo do percentual de participação operam de forma integrada: a presença é registrada uma única vez, códigos expirados são recusados e o histórico reflete imediatamente a nova participação.

Esses testes reforçam as características de Adequação Funcional, Segurança (confidencialidade e integridade) e Confiabilidade da norma ISO/IEC 25010, mapeadas no relatório de qualidade da Entrega 1. As mesmas rotas serão utilizadas no teste de carga do servidor, apresentado na seção seguinte.

O cenário de check-in offline com sincronização posterior, previsto na Entrega 1, depende do aplicativo Android em modo avião e não pode ser automatizado com Jest. Por isso, ele foi transferido para o teste de sistema/aceitação (Seção 2.4, tarefa 8).

2.4. Apresentar um Teste de Usuário (sistema/aceitação)

O teste de sistema/aceitação verifica se a solução como um todo atende aos requisitos do usuário final. A solução é composta pelo aplicativo Android, pela API publicada, pelo banco de dados e pelo módulo web administrativo usado no registro de presença. Diferentemente dos testes unitários e de integração, que são automatizados, este teste é feito por participantes do perfil do público-alvo. Eles executam tarefas reais no aplicativo instalado em um smartphone, sem ajuda da equipe.

Perfil do participante: serão convidados de um a três participantes na faixa etária e contexto dos usuários do próxima etapa. O perfil esperado é de jovens de 16 a 19 anos, cursando o ensino médio ou cursinho pré-vestibular, usuários de smartphone Android e sem contato prévio com o aplicativo. Participantes menores de 18 anos só participarão com autorização assinada pelo responsável legal, conforme o artigo 14 da LGPD.

Cenário de uso: após participar de uma palestra, o aluno instala o aplicativo e o usa pela primeira vez para acompanhar suas inscrições e apresentar o convite de presença no próximo encontro.

Procedimento: o teste segue o mesmo roteiro para todos os participantes, conforme as etapas abaixo.

- **Ambiente:** APK da versão final instalado em um smartphone Android e conectado à API publicada. O teste usa uma conta de aluno fictícia, com dados fictícios carregados: palestras inscritas, encontros, um certificado liberado e o teste de perfil. Um computador com o módulo web administrativo aberto fica disponível para a leitura do QR Code.
- **Condução:** um integrante atua como moderador e lê o roteiro padronizado. Outro registra as observações na ficha de cada participante, e faz o papel da equipe da Próxima Etapa, lendo o QR Code pelo módulo web. O participante é orientado a pensar em voz alta enquanto usa o aplicativo.
- **Sem ajuda:** o moderador não indica onde tocar nem como concluir a tarefa. Se o participante pedir ajuda ou ficar travado por mais de 2 minutos, a tarefa é registrada como “não concluída sem ajuda”, e o teste segue para a próxima.

- **Privacidade:** o teste não usa dados reais dos alunos. Nome, idade e escola dos participantes não são divulgados no relatório; eles são identificados apenas como P1, P2 e P3.

Critérios de aceitação por tarefa

Tarefa	Requisito	Critério de aceitação	Resultado (atende / não atende)
Fazer login	RFM01	Entra com e-mail e senha e chega à página inicial sem ajuda	Atende
Consultar cursos inscritos	RFM03	Vê apenas os cursos em que está inscrito, com carga horária e local	Atende
Ver o próximo encontro na agenda	RFM05	Encontra data, horário e local do próximo encontro	Atende
Registrar presença por QR Code	RFM06	Conclui o check-in em menos de 10 s e recebe confirmação	Atende
Consultar o histórico de presença	RFM07	Vê a presença registrada e o percentual de participação atualizado	Não Atende
Visualizar um certificado	RFM09	Abre o certificado disponível ou entende por que ainda não foi liberado	Atende
Responder ao teste de perfil	RFM10	Vê o resultado com o aviso "orientação, não diagnóstico psicológico"	Atende
Registrar presença sem internet	RFM15	A presença fica salva no celular e é sincronizada ao reconectar, uma única vez	Não Atende

Critério de aceitação geral: o participante conclui ao menos 90% das tarefas sem ajuda e abre o convite com o QR Code em menos de 10 segundos, conforme as metas definidas nas Seções 1.2 e 1.3. Como o teste tem oito tarefas, atingir 90% equivale a concluir todas as oito sem ajuda.

Resultados obtidos: os participantes concluíram a maior parte das tarefas com pouco esforço e relataram que o fluxo do aplicativo foi tranquilo de acompanhar. Login, consulta às palestras inscritas, agenda, apresentação do convite com QR Code, certificados e teste de perfil atenderam aos critérios de aceitação. Duas tarefas não atenderam: a consulta ao histórico de presença (tarefa 5) e a apresentação do

convite sem internet (tarefa 8). Com seis das oito tarefas atendidas (75%), o critério geral de 90% não foi atingido nesta rodada. As duas falhas são de funcionamento do sistema, e não de navegação, e já têm correção planejada, descrita a seguir.

Dificuldades encontradas:

- **Tarefa 5 – Histórico de presença:** a tela de Ingressos Anteriores não somava as palestras em que o participante havia comparecido. Por isso, o aluno não conseguia confirmar sua participação, mesmo com a presença registrada.
- **Tarefa 8 – Convite sem internet:** com o modo avião ativado, o QR Code não carregou. O aluno ficaria sem como apresentar o convite em locais com sinal fraco, situação comum na entrada de eventos.

Melhorias identificadas:

- **Corrigir a atualização do histórico de presença (prioridade alta):** garantir que cada presença confirmada pelo módulo web seja sincronizada com o aplicativo e apareça em Ingressos Anteriores, atendendo ao RFM07. Na mesma correção, incluir o número de encontros frequentados e o percentual de participação.
- **Disponibilizar o QR Code offline (prioridade alta):** gerar e salvar o QR Code no armazenamento local do celular (Room/SQLite) no momento da inscrição, para que o convite abra mesmo sem conexão, atendendo ao RFM15. Depois da correção, as tarefas 5 e 8 serão testadas novamente.
- **Onboarding de primeiro acesso (prioridade média):** telas curtas, que podem ser puladas, explicando para que serve cada ícone da barra inferior, o menu lateral e a assistente ProxIA. A melhoria reduz os erros de navegação no acesso às funções do menu lateral, identificados no teste de usabilidade.
- **Mensagem de certificado não liberado (prioridade média):** quando o certificado ainda não estiver disponível, informar o critério que falta, como a presença mínima, para que o aluno entenda o motivo sem precisar procurar a equipe.

2.5. Relatório de Teste de Carga do Servidor

O teste de carga verifica a característica Eficiência de Desempenho da ISO/IEC 25010, com a meta já definida na Seção 1.2.1: tempo de resposta das consultas e listagens abaixo de 2 segundos sob carga simulada. O cenário mais crítico do App do Aluno é o início de uma palestra, quando dezenas de alunos abrem o aplicativo e registram presença ao mesmo tempo.

Ferramenta e ambiente

Ferramenta: k6 (Grafana Labs), versão 0.49.0, com o script carga/teste-carga.js versionado no repositório.

Servidor: API em Node.js 22 com Express e banco Supabase, os mesmos dos testes de integração, executados na mesma máquina de teste.

Máquina: processador Intel Xeon 2,10 GHz com 2 vCPUs, 7,8 GB de memória RAM, sistema Linux. O k6 foi executado na mesma máquina, disputando processador com a API; por isso, os tempos medidos tendem a ser piores do que em um servidor dedicado.

Dados: banco populado por um seed de dados fictícios com 500 alunos, 20 cursos, 200 encontros (10 por curso) e 1.500 inscrições (3 cursos por aluno). O banco foi recriado antes de cada cenário.

Medição de recursos: uso de CPU e memória do processo da API amostrado a cada 2 segundos com o comando top durante toda a execução. Quando a API estiver em container, a mesma medição é feita com docker stats.

Cada usuário virtual representa um aluno fictício que faz login e, em seguida, repete ações sorteadas conforme os pesos do Quadro X, com intervalo de 0,5 a 1,5 segundo entre elas, simulando o tempo de leitura de uma pessoa real. No check-in, o script gera um QR Code válido, assinado da mesma forma que o módulo da ONG, para um encontro de um dos cursos do aluno. Quando o aluno já tinha presença naquele encontro, a API responde 409 (presença já registrada), que é o comportamento correto do sistema e, por isso, não é contado como erro.

Endpoints exercitados

Quadro X — Endpoints exercitados e peso nas requisições

Endpoint	Papel no cenário	Peso nas requisições
POST /auth/login	Aluno entra no aplicativo	10%
GET /alunos/me/cursos e GET /agenda	Consultas e listagens mais frequentes	60% (30% cada)
POST /presencas	Check-in por QR Code	25%
GET /presencas/historico	Conferência da presença	5%

Cenários:

Quadro Y — Cenários de carga

Cenário	Usuários virtuais (VUs)	Duração	O que simula
Carga normal	Sobe até 50 em 1 min e mantém	5 min	Uso cotidiano do aplicativo

Cenário	Usuários virtuais (VUs)	Duração	O que simula
Pico	De 0 a 200 em 30 s e mantém	2 min	Check-in simultâneo na abertura de um evento grande

Métricas e critérios de aprovação

Quadro Z — Métricas e critérios de aprovação

Métrica	Meta	Origem da meta
Tempo de resposta p95 das consultas e listagens	< 2 s	Seção 1.2.1 (Eficiência de Desempenho)
Tempo de resposta p95 do check-in (POST /presencas)	< 2 s	Mesma meta, aplicada à rota mais crítica
Taxa de erro (respostas 5xx ou timeout)	< 1%	Definida pelo grupo
Presenças duplicadas ao final do teste (consulta SQL)	0	Regra de presença única (Seção 1.2.2)
Uso de CPU e memória do container da API	Registrar	Acompanhamento com docker stats

As metas de tempo de resposta e de taxa de erro foram configuradas como limites automáticos (thresholds) no próprio script k6, que indica ao final se cada uma foi atingida. A verificação de presenças duplicadas é feita por uma consulta SQL ao banco logo após cada execução

Resultados:

Quadro W — Resultados do teste de carga

Métrica	Meta	Carga normal (50 VUs)	Pico (200 VUs)
p95 das consultas e listagens	< 2 s	19,5 ms — atende	76,0 ms — atende
p95 do check-in	< 2 s	21,8 ms — atende	106,0 ms — atende
Taxa de erro	< 1%	0,00% (0 de 16.392 requisições) — atende	0,00% (0 de 26.559 requisições) — atende

Métrica	Meta	Carga normal (50 VUs)	Pico (200 VUs)
Requisições por segundo	Registrar	45,4 req/s	175,4 req/s
Presenças duplicadas	0	0 (1.407 presenças gravadas) — atende	0 (3.949 presenças gravadas) — atende
CPU / memória da API	Registrar	CPU média 17%, máxima 32% de um núcleo; memória máxima 111 MB	CPU média 55%, máxima 80% de um núcleo; memória máxima 117 MB

Cada cenário foi executado duas vezes, com resultados consistentes entre as execuções. As Figuras X e Y apresentam o resumo de saída do k6 para cada cenário.

Figura X — Resumo do k6 no cenário de carga normal

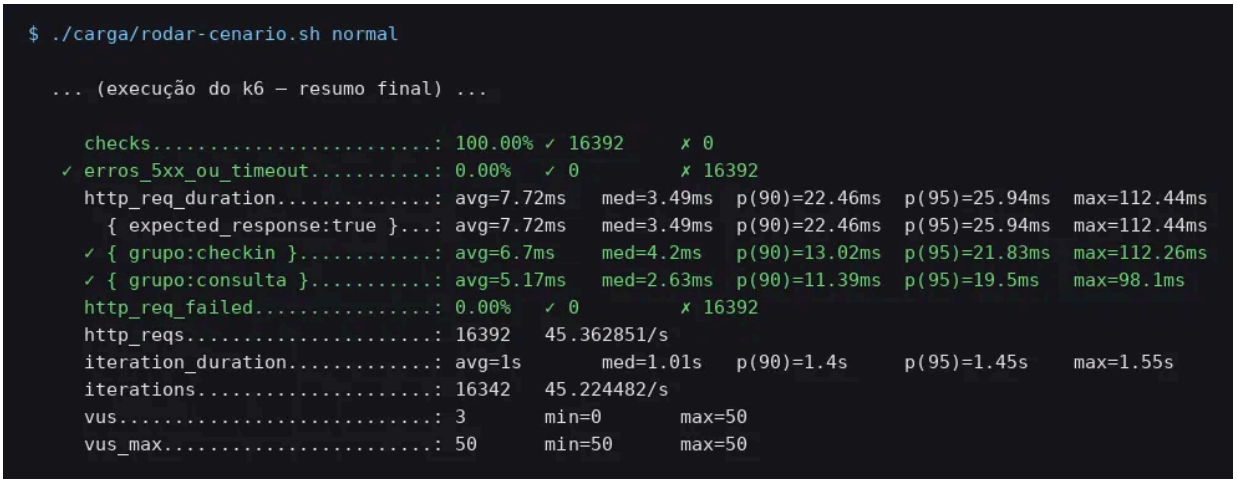


Figura Y — Resumo do k6 no cenário de pico

```
$ ./carga/rodar-cenario.sh pico

  ... (execução do k6 — resumo final) ...

     checks.........................: 100.00% ✓ 26559      ✗ 0
   ✓ erros_5xx_ou_timeout...........: 0.00%   ✓ 0          ✗ 26559
     http_req_duration..............: avg=25.52ms  med=16.46ms p(90)=64.44ms  p(95)=88.33ms  max=332.3ms
       { expected_response:true }...: avg=25.52ms  med=16.46ms p(90)=64.44ms  p(95)=88.33ms  max=332.3ms
     ✓ { grupo:checkin }............: avg=29.54ms  med=15.56ms p(90)=75.64ms  p(95)=105.99ms max=332.06ms
     ✓ { grupo:consulta }...........: avg=20.56ms  med=9.82ms  p(90)=54.82ms  p(95)=75.99ms  max=238.98ms
     http_req_failed................: 0.00%   ✓ 0          ✗ 26559
     http_reqs......................: 26559   175.351389/s
     iteration_duration.............: avg=1.02s    med=1.02s   p(90)=1.42s    p(95)=1.47s    max=1.75s
     iterations.....................: 26359   174.030922/s
     vus............................: 23      min=6        max=200
     vus_max........................: 200     min=200      max=200
```

Análise dos resultados:

Todas as metas foram atingidas nos dois cenários. Mesmo no pico, com 200 alunos simultâneos, o percentil 95 do tempo de resposta ficou em 76 ms nas consultas e 106 ms no check-in, cerca de vinte vezes abaixo do limite de 2 segundos definido na Seção 1.2.1. Nenhuma requisição retornou erro de servidor ou timeout em mais de 42 mil requisições somadas.

O check-in foi a rota mais lenta nos dois cenários, o que é esperado, pois é a única que grava no banco e realiza duas verificações antes da gravação: a validade do QR Code e a inscrição do aluno no curso. Ainda assim, a regra de presença única se manteve sob concorrência: ao final de cada cenário, a consulta ao banco não encontrou nenhuma presença duplicada, o que confirma que a restrição UNIQUE (aluno_id, encontro_id) protege a integridade dos dados mesmo com muitos check-ins simultâneos.

Quanto ao uso de recursos, o processador da API foi o ponto que mais cresceu com a carga: a média passou de 17% para 55% de um núcleo ao quadruplicar o número de usuários, com máxima de 80%, enquanto a memória permaneceu estável em torno de 110 MB. Como o Node.js executa as requisições em um único processo, esse é o gargalo previsto para cargas maiores, antes do banco de dados. Neste volume de dados, nenhuma consulta lenta foi identificada e não foi necessário criar índices adicionais: as buscas usam a chave primária de inscrição (aluno_id, curso_id) e o índice criado pela restrição de unicidade da presença.

Como melhorias para versões futuras, recomenda-se executar mais de uma instância da API (por exemplo, réplicas do container atrás de um balanceador) caso o número de alunos simultâneos cresça, e realizar o cenário de estresse no ambiente final de implantação, para identificar o limite real do servidor. Os resultados atendem às subcaracterísticas de comportamento temporal, utilização de recursos e capacidade da ISO/IEC 25010.

Apêndices

Roteiro do Moderador:

Antes do participante chegar: APK da versão final instalado e app fechado; dados do app limpos (primeiro acesso).

Conta fictícia criada (e-mail e senha anotados em um cartão para entregar ao participante).

QR Code do encontro gerado na hora e impresso ou exibido em outra tela (lembrar que ele expira em 15 minutos).

Um certificado liberado para a conta fictícia e o teste de perfil cadastrado.

Cronômetro pronto; fichas do Apêndice B impressas; autorização (Apêndice D) assinada.

Fala de abertura (ler em voz alta)

"Obrigado por participar. Vamos testar o aplicativo, não você. Não existe resposta certa ou errada. Vou pedir algumas tarefas e quero que você tente fazer sozinho, falando em voz alta o que está pensando. Eu não vou poder ajudar durante as tarefas, mas no final você pode perguntar o que quiser. Se em algum momento quiser parar, é só falar. Podemos começar?"

Tarefas (ler uma de cada vez, só passar para a próxima quando terminar)

1. "Imagine que você acabou de sair de uma palestra da Próxima Etapa e instalou o app. Entre na sua conta usando os dados deste cartão."
2. "Descubra em quais cursos você está inscrito e qual é a carga horária de um deles."
3. "Descubra quando e onde é o seu próximo encontro."
4. "Você chegou ao encontro. Registre sua presença usando este QR Code." (cronometrar)
5. "Confira se a sua presença foi registrada e qual é o seu percentual de participação."
6. "Veja se você já tem algum certificado e abra-o."
7. "Faça o teste de perfil e veja o seu resultado."
8. (Moderador ativa o modo avião.) "Agora você está sem internet. Registre a presença neste outro encontro." Depois: (desativar o modo avião) "Confira se a presença foi enviada."

Perguntas finais

- "O que foi mais fácil? E o mais difícil?"
- "Teve algum momento em que você ficou em dúvida sobre o que fazer?"
- "Você usaria esse app no lugar do Instagram para acompanhar a Próxima Etapa? Por quê?"

Referências Bibliográficas

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Geneva: ISO/IEC, 2023.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Education do Brasil, 2019.

FECAP. PI_3ADS_202602_Proxima_Etapa_FINAL.pdf — Documento de briefing do Projeto Integrador "Próxima Etapa — App do Aluno", 3º Semestre de Análise e Desenvolvimento de Sistemas, 2026.

GRAFANA LABS. k6 documentation. Disponível em: https://grafana.com/docs/k6/. Acesso em: [inserir data].

JEST. Jest documentation. Disponível em: https://jestjs.io/docs/getting-started. Acesso em: [inserir data].

SUPERTEST. Supertest: HTTP assertions made easy via superagent. Disponível em: https://github.com/ladjs/supertest. Acesso em: [inserir data].